



OPERACIONES BÁSICAS CON HIBERNATE

Operaciones Básicas con Hibernate usando el Patrón DAO - Insertar

Introducción

En esta lección implementaremos el método `insertar` en nuestra clase `PersonaDAO` para guardar objetos `Persona` en la base de datos, aplicando buenas prácticas y simplificación con `try-with-resources`. También probaremos el método desde la clase `OperacionesHibernateJPA`.

1. Modificar la clase `PersonaDAO.java`

 Ruta: `src/main/java/gm/dao/PersonaDAO.java`

Explicación del cambio

Agregamos el método `insertar(Persona persona)`, en el cual se abre una transacción, se ejecuta la operación de persistencia y se hace `commit`. En caso de error, se realiza un `rollback`.

👉 El método no cierra el `EntityManager`, ya que eso se realiza automáticamente en el método `close()` de la clase, gracias a la implementación de `AutoCloseable`.

Código actualizado

```
package gm.dao;

import gm.domain.Persona;
import jakarta.persistence.*;

import java.util.List;

public class PersonaDAO implements AutoCloseable {

    private final EntityManagerFactory emf;
    private final EntityManager em;

    public PersonaDAO() {
        this.emf = Persistence.createEntityManagerFactory("HibernatePU");
        this.em = emf.createEntityManager();
    }

    public List<Persona> listar() {
        String hql = "SELECT p FROM Persona p";
        return em.createQuery(hql, Persona.class).getResultList();
    }

    public void imprimirPersonas() {
        listar().forEach(p -> System.out.println(" " + p));
    }

    public void insertar(Persona persona) {
        var tx = em.getTransaction();
        try {
            tx.begin();
            em.persist(persona);
            tx.commit();
        } catch (Exception e) {
            if (tx.isActive()) {
                tx.rollback();
            }
            throw e; // se relanza la excepción para que la maneje el llamador si quiere
        }
    }
}
```

```
    }  
}  
  
@Override  
public void close() {  
    if (em.isOpen()) {  
        em.close();  
    }  
    if (emf.isOpen()) {  
        emf.close();  
    }  
}  
}
```



2. Probar el método insertar

📄 Ruta: `src/main/java/test/OperacionesHibernateJPA.java`

Explicación

En esta clase vamos a crear un objeto `Persona`, lo inicializamos con valores y lo insertamos usando nuestro DAO.

Código final

```
package test;  
  
import gm.dao.PersonaDAO;  
import gm.domain.Persona;  
  
public class OperacionesHibernateJPA {  
  
    public static void main(String[] args) {  
        try (PersonaDAO personaDao = new PersonaDAO()) {  
            // 1. Caso listar  
            personaDao.imprimirPersonas();  
  
            // 2. Caso insertar  
            Persona persona = new Persona();  
            persona.setNombre("Carlos");  
            persona.setApellido("Lara");  
            persona.setEmail("clara@mail.com");  
        }  
    }  
}
```

```
        persona.setTelefono("55443322");

        personaDao.insertar(persona);
    }

}

}
```

💡 Nota sobre la llave primaria

La clave primaria (`idPersona`) se genera automáticamente porque está definida como `AUTO_INCREMENT` en MySQL y está correctamente anotada en la entidad `Persona.java`.

No es necesario asignarla manualmente en Java; Hibernate se encarga de generarla al momento de persistir el objeto.

✅ Resultado esperado

Al ejecutar la clase `OperacionesHibernateJPA`, deberías ver en la consola el listado de personas antes y después de la inserción. El nuevo registro aparecerá con un nuevo `idPersona`.

```
14:43:16.030 [main] DEBUG org.hibernate.SQL - select
p1_0.id_persona,p1_0.apellido,p1_0.email,p1_0.nombre,p1_0.telefono from persona p1_0

Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}

Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}

Persona{idPersona=3, nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}

Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}

14:43:16.129 [main] DEBUG org.hibernate.SQL - insert into persona (apellido,email,nombre,telefono) values
(?,?,?,?)

14:43:16.132 [main] TRACE org.hibernate.orm.jdbc.bind - binding parameter (1:VARCHAR) <- [Lara]

14:43:16.133 [main] TRACE org.hibernate.orm.jdbc.bind - binding parameter (2:VARCHAR) <- [lara@mail.com]

14:43:16.133 [main] TRACE org.hibernate.orm.jdbc.bind - binding parameter (3:VARCHAR) <- [Carlos]

14:43:16.133 [main] TRACE org.hibernate.orm.jdbc.bind - binding parameter (4:VARCHAR) <- [55443322]
```

🎉 Conclusión

Hemos implementado correctamente el método `insertar()` en nuestro patrón DAO y lo probamos desde la clase `OperacionesHibernateJPA`. Esta estructura es limpia, modular y lista para ser extendida con operaciones como actualizar y eliminar.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)